

RUBINA DANGOL MAHARJAN AI FELLOWSHIP FUSEMACHINE



4th May 2026

Week-1 Assignment

SQL Fundamentals: Practice cleaning, joining and aggregating data using the classic Models retail database.

Presented to

Pralisha Kansakar

Presented by

Rubina Maharjan

TABLE OF CONTENTS

About the assignment	
Query Execution Summary	
Assignment	
SQL Error journal	
Overall results	
Possible improvements	
Conclusion	

ABOUT

Assignment Overview

This SQL assignment was completed using the **Classic Models** sample database. The main objective was to apply SQL concepts such as **filtering, joins, aggregation, subqueries, CTE/WITH clause, ranking, views, update, delete, and insert operations**. The uploaded assignment document lists 20 tasks, including customer filtering, employee reporting, office-wise sales/payment reports, view creation, update, delete, and manual payment insertion.

Database Used:

Database name: classicmodels

The database was selected using:

```
USE classicmodels;
```

Files:

- 'mysqlsampledatabase.sql' - The database dump to import
- 'classic_models_practice_solutions.sql' - The SQL solution

Query Execution Summary

MySQL Workbench action output

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
1	13:05:29	SELECT * FROM customers WHERE creditLimit > 20000 LIMIT 0, 1000	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH...	0.015 sec
2	13:05:38	USE classicmodels	0 row(s) affected	0.032 sec
3	13:05:38	SELECT * FROM customers WHERE creditLimit > 20000 LIMIT 0, 1000	97 row(s) returned	0.031 sec / 0.000 sec
4	13:05:38	SELECT e.* FROM employees e JOIN employees vp ON e.reportsTo = vp.employeeNumber WHERE vp.jobTitle...	4 row(s) returned	0.016 sec / 0.000 sec
5	13:05:38	SELECT * FROM customers WHERE state IS NOT NULL AND country = 'USA' AND creditLimit BETWEEN 1...	8 row(s) returned	0.000 sec / 0.000 sec
6	13:05:38	SELECT e.* FROM employees e JOIN employees m ON e.reportsTo = m.employeeNumber WHERE m.jobTitle LI...	15 row(s) returned	0.000 sec / 0.000 sec
7	13:05:38	SELECT country, AVG(creditLimit) AS averageCreditLimit FROM customers GROUP BY country LIMIT 0, 1000	29 row(s) returned	0.000 sec / 0.000 sec
8	13:05:38	SELECT orderDate, customerNumber, COUNT(*) AS totalOrders FROM orders GROUP BY orderDate, customer...	0 row(s) returned	0.000 sec / 0.000 sec
9	13:05:38	SELECT CONCAT(firstName, ' ', lastName) AS supervisorName, jobTitle AS supervisorJobTitle, (SELECT CO...	6 row(s) returned	0.000 sec / 0.000 sec
10	13:05:38	SELECT CONCAT(firstName, ' ', lastName) AS supervisorName, s.jobTitle AS supervisorJobTitle, COUNT(...	6 row(s) returned	0.000 sec / 0.000 sec
11	13:05:38	WITH avg_credit AS (SELECT AVG(creditLimit) AS avgCreditLimit FROM customers) SELECT c.* FROM cus...	69 row(s) returned	0.000 sec / 0.000 sec
12	13:05:38	WITH ranked_customers AS (SELECT customerNumber, customerName, creditLimit, RANK() OVER ...	122 row(s) returned	0.015 sec / 0.000 sec
13	13:05:38	WITH ranked_customers AS (SELECT customerNumber, customerName, creditLimit, DENSE_RAN...	1 row(s) returned	0.000 sec / 0.000 sec
14	13:05:38	SELECT officeCode, COUNT(*) AS totalEmployees FROM employees GROUP BY officeCode LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec
15	13:05:38	SELECT e.officeCode, COUNT(c.customerNumber) AS totalCustomers FROM offices o JOIN employees e ON o...	7 row(s) returned	0.000 sec / 0.000 sec
16	13:05:38	SELECT o.officeCode, o.city AS officeName, o.state, o.country, SUM(p.amount) AS totalPaymentsReceiv...	7 row(s) returned	0.000 sec / 0.000 sec
17	13:05:38	SELECT o.officeCode, o.city AS officeName, o.state, o.country, SUM(od.quantityOrdered * od.priceEach) ...	7 row(s) returned	0.015 sec / 0.000 sec
18	13:05:39	WITH payments_by_customer AS (SELECT customerNumber, SUM(amount) AS totalPaid FROM payments ...	7 row(s) returned	0.016 sec / 0.000 sec
19	13:05:39	SELECT customerNumber, customerName, country, creditLimit, creditLimit / SUM(creditLimit) OVER (PAR...	122 row(s) returned	0.000 sec / 0.000 sec
20	13:05:39	CREATE OR REPLACE VIEW customer_order_summary AS SELECT c.customerNumber, c.customerName, ...	0 row(s) affected	0.062 sec
21	13:05:39	UPDATE customers SET country = 'United States' WHERE customerNumber = 103	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0	0.000 sec
22	13:05:39	DELETE FROM payments WHERE amount < 20000	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses a...	0.000 sec

Step 1: Open a terminal and log in to MySQL

Use root account or another account with permission to create databases and load data.

```
mysql -u root -p
```

Step 2: Create the Classic Models database if needed and use the classic models

```
USE classicmodels;
```

Step 3: Import the dump file

-sql:

"source C:/Users/rubin/OneDrive/Desktop/AI fellow/week1/sql-fundamentals/mysqlsampledatabase.sql"

Step 4: Confirm the tables loaded correctly

```
mysql> SHOW TABLES;
+-----+
| Tables_in_classicmodels |
+-----+
| customers                |
| employees                |
| offices                  |
| orderdetails             |
| orders                   |
| payments                 |
| productlines             |
| products                 |
+-----+
8 rows in set (0.00 sec)
```

```
mysql> DESC customers;
```

Field	Type	Null	Key	Default	Extra
customerNumber	int	NO	PRI	NULL	
customerName	varchar(50)	NO		NULL	
contactLastName	varchar(50)	NO		NULL	
contactFirstName	varchar(50)	NO		NULL	
phone	varchar(50)	NO		NULL	
addressLine1	varchar(50)	NO		NULL	
addressLine2	varchar(50)	YES		NULL	
city	varchar(50)	NO		NULL	
state	varchar(50)	YES		NULL	
postalCode	varchar(15)	YES		NULL	
country	varchar(50)	NO		NULL	
salesRepEmployeeNumber	int	YES	MUL	NULL	
creditLimit	decimal(10,2)	YES		NULL	

13 rows in set (0.00 sec)

```
mysql> DESC orders;
```

Field	Type	Null	Key	Default	Extra
orderNumber	int	NO	PRI	NULL	
orderDate	date	NO		NULL	
requiredDate	date	NO		NULL	
shippedDate	date	YES		NULL	
status	varchar(15)	NO		NULL	
comments	text	YES		NULL	
customerNumber	int	NO	MUL	NULL	

7 rows in set (0.01 sec)

```
mysql> DESC employees;
```

Field	Type	Null	Key	Default	Extra
employeeNumber	int	NO	PRI	NULL	
lastName	varchar(50)	NO		NULL	
firstName	varchar(50)	NO		NULL	
extension	varchar(10)	NO		NULL	
email	varchar(100)	NO		NULL	
officeCode	varchar(10)	NO	MUL	NULL	
reportsTo	int	YES	MUL	NULL	
jobTitle	varchar(50)	NO		NULL	

8 rows in set (0.09 sec)

```
mysql> DESC offices;
```

Field	Type	Null	Key	Default	Extra
officeCode	varchar(10)	NO	PRI	NULL	
city	varchar(50)	NO		NULL	
phone	varchar(50)	NO		NULL	
addressLine1	varchar(50)	NO		NULL	
addressLine2	varchar(50)	YES		NULL	
state	varchar(50)	YES		NULL	
country	varchar(50)	NO		NULL	
postalCode	varchar(15)	NO		NULL	
territory	varchar(10)	NO		NULL	

9 rows in set (0.00 sec)

```
mysql> DESC orderdetails;
```

Field	Type	Null	Key	Default	Extra
orderNumber	int	NO	PRI	NULL	
productCode	varchar(15)	NO	PRI	NULL	
quantityOrdered	int	NO		NULL	
priceEach	decimal(10,2)	NO		NULL	
orderLineNumber	smallint	NO		NULL	

5 rows in set (0.00 sec)

```
mysql> DESC payments;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| customerNumber | int           | NO   | PRI | NULL    |       |
| checkNumber    | varchar(50)   | NO   | PRI | NULL    |       |
| paymentDate    | date          | NO   |     | NULL    |       |
| amount         | decimal(10,2) | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Step 5: Run the practice script

Open `classic\_models\_practice\_solutions.sql` in MySQL Workbench or run it from the MySQL prompt.

SQL code:

“source C:/Users/rubin/OneDrive/Desktop/AI fellow/week1/sql-fundamentals/classic_models_practice_solutions.sql;”

Assignment

The solution script follows the same pattern as the assignment tasks

Task 1: Show all the customers whose creditLimit is greater than 20000 - 97 rows returned

```
mysql> SELECT *
-> FROM customers
-> WHERE creditLimit > 20000;
```

customerNumber	customerName	contactLastName	contactFirstName	creditLimit
103	Atelier graphique	Schmitt	Carine	21000.00
112	Signal Gift Stores	King	Jean	71800.00
114	Australian Collectors, Co.	Ferguson	Peter	117300.00
119	La Rochelle Gifts	Labrun	Janine	118200.00
121	Baane Mini Imports	Bergulfsen	Jonas	81700.00
124	Mini Gifts Distributors Ltd.	Nelson	Susan	210500.00
128	Blauer See Auto, Co.	Keitel	Roland	59700.00
129	Mini Wheels Co.	Murphy	Julie	64600.00

Task 2: Show the employees who report to VP Sales - 4 rows returned

```
mysql> SELECT e.*
-> FROM employees e
-> JOIN employees vp ON e.reportsTo = vp.employeeNumber
-> WHERE vp.jobTitle = 'VP Sales';
```

employeeNumber	lastName	firstName	extension	email	officeCode
1088	Patterson	William	x4871	wpatterson@classicmodelcars.com	6
1102	Bondur	Gerard	x5408	gbondur@classicmodelcars.com	4
1143	Bow	Anthony	x5428	abow@classicmodelcars.com	1
1621	Nishi	Mami	x101	mnishi@classicmodelcars.com	5

Task 3: Find all the customers who have set their state while filling the forms and Lives in USA and credit limit is between 100000 and 200000 - 8 rows returned

```
mysql> SELECT *
-> FROM customers
-> WHERE state IS NOT NULL
-> AND country = 'USA'
-> AND creditLimit BETWEEN 100000 AND 200000;
```

customerNumber	customerName	contactLastName	contactFirstName	phone	address
131	Land of Toys Inc.	Lee	Kwai	2125557818	897 L
151	Muscle Machine Inc	Young	Jeff	2125557413	4092
157	Diecast Classics Inc.	Leong	Kelvin	2155551555	7586
239	Collectable Mini Designs Co.	Thompson	Valarie	7605558146	361 F
286	Marta's Replicas Co.	Hernandez	Marta	6175558555	39323
319	Mini Classics	Frick	Steve	9145554562	3758

Task 4: Find all the employees who report to Sales Managers of all types
- 15 rows returned

```
mysql> SELECT e.*
-> FROM employees e
-> JOIN employees m ON e.reportsTo = m.employeeNumber
-> WHERE m.jobTitle LIKE '%Sales Manager%'
-> OR m.jobTitle LIKE '%Sale Manager%';
```

employeeNumber	lastName	firstName	extension	email
1611	Fixter	Andy	x101	afixter@classicmodelcars.com
1612	Marsh	Peter	x102	pmarsh@classicmodelcars.com
1619	King	Tom	x103	tking@classicmodelcars.com
1337	Bondur	Loui	x6493	lbondur@classicmodelcars.com
1370	Hernandez	Gerard	x2028	ghernande@classicmodelcars.com
1401	Castillo	Pamela	x2759	pcastillo@classicmodelcars.com
1501	Bott	Larry	x2311	lbott@classicmodelcars.com
1504	Jones	Barry	x102	bjones@classicmodelcars.com
1702	Gerard	Martin	x2312	mgerard@classicmodelcars.com
1165	Jennings	Leslie	x3291	ljennings@classicmodelcars.com
1166	Thompson	Leslie	x4065	lthompson@classicmodelcars.com
1188	Firrelli	Julie	x2173	jfirrelli@classicmodelcars.com
1216	Patterson	Steve	x4334	spatterson@classicmodelcars.com
1286	Tseng	Foon Yue	x2248	ftseng@classicmodelcars.com
1323	Vanauf	George	x4102	gvanauf@classicmodelcars.com

15 rows in set (0.00 sec)

Task 5: Find the average credit limit of customers of each country - 29 rows returned

```
mysql> SELECT country, AVG(creditLimit) AS averageCreditLimit
-> FROM customers
-> GROUP BY country;
```

country	averageCreditLimit
United States	21000.000000
USA	78102.777778
Australia	86060.000000
France	82845.454545
Norway	81700.000000
Poland	0.000000
Germany	19776.923077
Spain	73971.428571
Sweden	84750.000000
Denmark	102100.000000
Singapore	67233.333333
Norway	95950.000000

Task 6: Find the total no. of orders for each date and customer. Show only dates with total number of orders greater than 10 for date and customer - 0 rows returned

```
mysql> SELECT orderDate, customerNumber, COUNT(*) AS totalOrders
-> FROM orders
-> GROUP BY orderDate, customerNumber
-> HAVING COUNT(*) > 10;
Empty set (0.01 sec)
```

Task 7: Find the name of the supervisor, job title of supervisor and total no. of supervisee using subquery. (Without using Join operation) - 6 rows returned

```
mysql> SELECT
->  CONCAT(firstName, ' ', lastName) AS supervisorName,
->  jobTitle AS supervisorJobTitle,
->  (SELECT COUNT(*)
->   FROM employees e
->   WHERE e.reportsTo = s.employeeNumber) AS totalSupervisee
-> FROM employees s
-> WHERE EXISTS (
->  SELECT 1
->  FROM employees e
->  WHERE e.reportsTo = s.employeeNumber
-> );
```

supervisorName	supervisorJobTitle	totalSupervisee
Diane Murphy	President	2
Mary Patterson	VP Sales	4
William Patterson	Sales Manager (APAC)	3
Gerard Bondur	Sale Manager (EMEA)	6
Anthony Bow	Sales Manager (NA)	6
Mami Nishi	Sales Rep	1

6 rows in set (0.01 sec)

Task 8: Find the name of the supervisor, job title of supervisor and total no. of supervisee using subquery. (With using Join operation)

```
mysql> SELECT
->  CONCAT(s.firstName, ' ', s.lastName) AS supervisorName,
->  s.jobTitle AS supervisorJobTitle,
->  COUNT(e.employeeNumber) AS totalSupervisee
-> FROM employees s
-> JOIN employees e ON e.reportsTo = s.employeeNumber
-> GROUP BY s.employeeNumber, s.firstName, s.lastName, s.jobTitle;
```

supervisorName	supervisorJobTitle	totalSupervisee
Diane Murphy	President	2
Mary Patterson	VP Sales	4
William Patterson	Sales Manager (APAC)	3
Gerard Bondur	Sale Manager (EMEA)	6
Anthony Bow	Sales Manager (NA)	6
Mami Nishi	Sales Rep	1

6 rows in set (0.00 sec)

Task 9: Find all customers with a credit limit greater than average credit limit using WITH Clause.

```
mysql> WITH avg_credit AS (
->  SELECT AVG(creditLimit) AS avgCreditLimit
->  FROM customers
-> )
-> SELECT c.*
-> FROM customers c
-> CROSS JOIN avg_credit a
-> WHERE c.creditLimit > a.avgCreditLimit;
```

customerNumber	customerName	city	state	contactLastName	contactFirstName	phoneArea	phone
112	Signal Gift Stores	Las Vegas	NV	King	Jean	83030	702
114	Australian Collectors, Co.	Melbourne	Victoria	Ferguson	Peter	3004	03
119	La Rochelle Gifts	Nantes	NULL	Labrune	Janine	44000	40
121	Baane Mini Imports	Bergulfen	Jonas				07

486	Motor Mint Distributors Inc.	Philadelphia	PA	Salazar	Rosa	2155559857	1216	85000.00
495	Diecast Collectables	Boston	MA	Franco	Valarie	6175552555	1323	72600.00
496	Kelly's Gift Shop	Auckland	NULL	Snowden	Tony	+64 9 5555500	1188	85100.00
							1612	110000.00

69 rows in set (0.00 sec)

Task 10: Find the rank of customer. Then, find the customer with the third highest credit limit. 123 rows returned

```
mysql> WITH ranked_customers AS (  
-> SELECT  
->   customerNumber,  
->   customerName,  
->   creditLimit,  
->   RANK() OVER (ORDER BY creditLimit DESC) AS credit_rank  
-> FROM customers  
-> )  
-> SELECT *  
-> FROM ranked_customers  
-> ORDER BY credit_rank, customerName;
```

customerNumber	customerName	creditLimit	credit_rank
141	Euro+ Shopping Channel	227600.00	1
124	Mini Gifts Distributors Ltd.	210500.00	2
298	Vida Sport, Ltd	141300.00	3
151	Muscle Machine Inc	138500.00	4
187	AV Stores, Co.	136800.00	5
146	Saveley & Henriot, Co.	123900.00	6
286	Marta's Replicas Co.	123700.00	7
386	L'ordine Souvenirs	121400.00	8
227	Heintze Collectables	120800.00	9
259	Toms Spezialitäten, Ltd	120400.00	10
278	Rovelli Gifts	119600.00	11
119	La Rochelle Gifts	118200.00	12
114	Australian Collectors, Co.	117300.00	13
448	Scandinavian Gift Ideas	116400.00	14
131	Land of Toys Inc.	114900.00	15
363	Online Diecast Creations Co.	114200.00	16

```
mysql>  
mysql> WITH ranked_customers AS (  
-> SELECT  
->   customerNumber,  
->   customerName,  
->   creditLimit,  
->   DENSE_RANK() OVER (ORDER BY creditLimit DESC) AS credit_rank  
-> FROM customers  
-> )  
-> SELECT *  
-> FROM ranked_customers  
-> WHERE credit_rank = 3;
```

customerNumber	customerName	creditLimit	credit_rank
298	Vida Sport, Ltd	141300.00	3

1 row in set (0.00 sec)

Task 11: Generate a report that shows total no. of employees working in each office.

```
mysql> SELECT officeCode, COUNT(*) AS totalEmployees  
-> FROM employees  
-> GROUP BY officeCode;
```

officeCode	totalEmployees
1	6
2	2
3	2
4	5
5	2
6	4
7	2

7 rows in set (0.00 sec)

Task 12: Generate a report that shows total no. of customers visited each office.

```
mysql> SELECT e.officeCode, COUNT(c.customerNumber) AS totalCustomers  
-> FROM offices o
```

```

-> JOIN employees e ON o.officeCode = e.officeCode
-> LEFT JOIN customers c ON c.salesRepEmployeeNumber = e.employeeNumber
-> GROUP BY e.officeCode
-> ORDER BY e.officeCode;
+-----+-----+
| officeCode | totalCustomers |
+-----+-----+
| 1          | 12             |
| 2          | 12             |
| 3          | 15             |
| 4          | 29             |
| 5          | 5              |
| 6          | 10             |
| 7          | 17             |
+-----+-----+
7 rows in set (0.01 sec)

```

Task 13: Generate a report that shows total payment received by each office using payment tables and essential tables.

```

mysql> SELECT
-> o.officeCode,
-> o.city AS officeName,
-> o.state,
-> o.country,
-> SUM(p.amount) AS totalPaymentsReceived
-> FROM offices o
-> JOIN employees e ON e.officeCode = o.officeCode
-> JOIN customers c ON c.salesRepEmployeeNumber = e.employeeNumber
-> JOIN payments p ON p.customerNumber = c.customerNumber
-> GROUP BY o.officeCode, o.city, o.state, o.country
-> ORDER BY o.officeCode;
+-----+-----+-----+-----+-----+
| officeCode | officeName | state | country | totalPaymentsReceived |
+-----+-----+-----+-----+-----+
| 1          | San Francisco | CA    | USA     | 1337439.58           |
| 2          | Boston       | MA    | USA     | 835882.33            |
| 3          | NYC          | NY    | USA     | 1072619.47           |
| 4          | Paris        | NULL  | France  | 2819168.90           |
| 5          | Tokyo        | Chiyoda-Ku | Japan   | 457110.07            |
| 6          | Sydney       | NULL  | Australia | 1007292.98           |
| 7          | London       | NULL  | UK      | 1324325.90           |
+-----+-----+-----+-----+-----+
7 rows in set (0.01 sec)

```

Task 14: Generate a report that shows total sales(in amount) by each office using order details table and other essential tables.

```

mysql> SELECT
-> o.officeCode,
-> o.city AS officeName,
-> o.state,
-> o.country,
-> SUM(od.quantityOrdered * od.priceEach) AS totalSalesAmount
-> FROM offices o
-> JOIN employees e ON e.officeCode = o.officeCode
-> JOIN customers c ON c.salesRepEmployeeNumber = e.employeeNumber
-> JOIN orders ord ON ord.customerNumber = c.customerNumber
-> JOIN orderdetails od ON od.orderNumber = ord.orderNumber
-> GROUP BY o.officeCode, o.city, o.state, o.country
-> ORDER BY o.officeCode;
+-----+-----+-----+-----+-----+
| officeCode | officeName | state | country | totalSalesAmount |
+-----+-----+-----+-----+-----+
| 1          | San Francisco | CA    | USA     | 1429063.57         |
| 2          | Boston       | MA    | USA     | 892538.62          |
| 3          | NYC          | NY    | USA     | 1157589.72         |
| 4          | Paris        | NULL  | France  | 3083761.58         |
| 5          | Tokyo        | Chiyoda-Ku | Japan   | 457110.07          |
| 6          | Sydney       | NULL  | Australia | 1147176.35         |
| 7          | London       | NULL  | UK      | 1436950.70         |
+-----+-----+-----+-----+-----+
7 rows in set (0.01 sec)

```

Task 15: Generate a report that shows total payment pending for each office.

```
mysql> WITH payments_by_customer AS (
```

```

mysql> WITH payments_by_customer AS (
-> SELECT customerNumber, SUM(amount) AS totalPaid
-> FROM payments
-> GROUP BY customerNumber
-> ),
-> sales_by_customer AS (
-> SELECT customerNumber, SUM(quantityOrdered * priceEach) AS totalSales
-> FROM orders ord
-> JOIN orderdetails od ON od.orderNumber = ord.orderNumber
-> GROUP BY customerNumber
-> )
-> SELECT
-> o.officeCode,
-> o.city AS officeName,
-> o.state,
-> o.country,
-> SUM(COALESCE(s.totalSales, 0) - COALESCE(p.totalPaid, 0)) AS totalPaymentPending
-> FROM offices o
-> JOIN employees e ON e.officeCode = o.officeCode
-> JOIN customers c ON c.salesRepEmployeeNumber = e.employeeNumber
-> LEFT JOIN sales_by_customer s ON s.customerNumber = c.customerNumber
-> LEFT JOIN payments_by_customer p ON p.customerNumber = c.customerNumber
-> GROUP BY o.officeCode, o.city, o.state, o.country
-> ORDER BY o.officeCode;

```

	officeCode	officeName	state	country	totalPaymentPending
1	1	San Francisco	CA	USA	91623.99
2	2	Boston	MA	USA	56656.29
3	3	NYC	NY	USA	84970.25
4	4	Paris	NULL	France	264592.68
5	5	Tokyo	Chiyoda-Ku	Japan	0.00
6	6	Sydney	NULL	Australia	139883.37
7	7	London	NULL	UK	112624.80

7 rows in set (0.01 sec)

Task 16: Find the creditLimit of each person, proportion of creditLimit of each person in each country. 122 rows returned

```

mysql> SELECT
-> customerNumber,
-> customerName,
-> country,
-> creditLimit,
-> creditLimit / SUM(creditLimit) OVER (PARTITION BY country) AS creditLimitProportion
-> FROM customers
-> ORDER BY country, customerName;

```

	customerNumber	customerName	country	creditLimit	creditLimitProportion
276	276	Anna's Decorations, Ltd	Australia	107800.00	0.250523
471	471	Australian collectables, Ltd	Australia	60300.00	0.140135
114	114	Australian Collectors, Co.	Australia	117300.00	0.272601
333	333	Australian Gift Network, Co	Australia	51600.00	0.119916
282	282	Souvenirs And Things Co.	Australia	93300.00	0.216825
452	452	Mini Auto Werke	Austria	45300.00	0.387179
382	382	Salzburg Collectables	Austria	71700.00	0.612821
314	314	Petit Auto	Belgium	79900.00	0.772727
381	381	Royale Belge	Belgium	23500.00	0.227273
202	202	Canadian Gift Exchange Network	Canada	90300.00	0.395013
233	233	Quebec Home Shopping Network	Canada	48700.00	0.213036
260	260	Royal Canadian Collectables, Ltd.	Canada	89600.00	0.391951

Task 17: Create a view showing the customer name, complete address, and their total number of orders.

```

mysql> CREATE OR REPLACE VIEW customer_order_summary AS
-> SELECT
-> c.customerNumber,
-> c.customerName,
-> CONCAT_WS(' ', c.addressLine1, c.addressLine2, c.city, c.state, c.postalCode, c.country) AS completeAddress,
-> COUNT(o.orderNumber) AS totalOrders
-> FROM customers c
-> LEFT JOIN orders o ON o.customerNumber = c.customerNumber
-> GROUP BY c.customerNumber, c.customerName, c.addressLine1, c.addressLine2, c.city, c.state, c.postalCode, c.country;
Query OK, 0 rows affected (0.05 sec)

```

Task 18: Update the country of a customer (use any one record).

```

mysql> UPDATE customers
-> SET country = 'United States'
-> WHERE customerNumber = 103;
Query OK, 0 rows affected (0.01 sec)

```

```
Rows matched: 1  Changed: 0  Warnings: 0
```

Task 19: Delete all payments below 20,000.

```
mysql> DELETE FROM payments
-> WHERE amount < 20000;
Query OK, 78 rows affected (0.01 sec)
```

Task 20: Add new payments manually for an existing customer.

```
mysql> INSERT INTO payments (customerNumber, checkNumber, paymentDate, amount)
-> VALUES (103, 'NEW0001', '2026-05-04', 25000.00);
Query OK, 1 row affected (0.01 sec)
```

Use CASE statements and filtering to clean data.

```
mysql> SELECT
-> customerName,
-> creditLimit,
-> CASE
-> WHEN creditLimit >= 100000 THEN 'High Credit Limit'
-> WHEN creditLimit >= 50000 THEN 'Medium Credit Limit'
-> ELSE 'Low Credit Limit'
-> END AS credit_category
-> FROM customers;
```

customerName	creditLimit	credit_category
Atelier graphique	21000.00	Low Credit Limit
Signal Gift Stores	71800.00	Medium Credit Limit
Australian Collectors, Co.	117300.00	High Credit Limit
La Rochelle Gifts	118200.00	High Credit Limit
Baane Mini Imports	81700.00	Medium Credit Limit
Mini Gifts Distributors Ltd.	210500.00	High Credit Limit
Havel & Zbyszek Co	0.00	Low Credit Limit
Blauer See Auto, Co.	59700.00	Medium Credit Limit
Mini Wheels Co.	64600.00	Medium Credit Limit
Land of Toys Inc.	114900.00	High Credit Limit
Euro+ Shopping Channel	227600.00	High Credit Limit
Volvo Model Replicas, Co	53100.00	Medium Credit Limit
Danish Wholesale Imports	83400.00	Medium Credit Limit
Saveley & Henriot, Co.	123900.00	High Credit Limit
Dragon Souvenirs, Ltd.	103800.00	High Credit Limit
Muscle Machine Inc.	138500.00	High Credit Limit
Diecast Classics Inc.	100600.00	High Credit Limit
Technics Stores Inc.	84600.00	Medium Credit Limit
Handji Gifts& Co	97900.00	Medium Credit Limit

Microscale Inc.	39800.00	Low Credit Limit
Corrida Auto Replicas, Ltd	104600.00	High Credit Limit
Warburg Exchange	0.00	Low Credit Limit
FunGiftIdeas.com	85800.00	Medium Credit Limit
Anton Designs, Ltd.	0.00	Low Credit Limit
Australian Collectables, Ltd	60300.00	Medium Credit Limit
Frau da Collezione	34800.00	Low Credit Limit
West Coast Collectables Co.	55400.00	Medium Credit Limit
Mit Vergnügen & Co.	0.00	Low Credit Limit
Kremlin Collectables, Co.	0.00	Low Credit Limit
Raanan Stores, Inc	0.00	Low Credit Limit
Iberia Gift Imports, Corp.	65700.00	Medium Credit Limit
Motor Mint Distributors Inc.	72600.00	Medium Credit Limit
Signal Collectibles Ltd.	60300.00	Medium Credit Limit
Double Decker Gift Stores, Ltd	43300.00	Low Credit Limit
Diecast Collectables	85100.00	Medium Credit Limit
Kelly's Gift Shop	110000.00	High Credit Limit

122 rows in set (0.07 sec)

SQL error journal

Error	Cause	Solution
mysql not recognized	MySQL path was not added to environment variables	Used full MySQL path or added MySQL bin folder to PATH
Unknown database classicmodels	Database was not imported yet	Ran source mysqlsampledatabase.sql;
Unknown column	Wrong column name used	Checked table structure using DESC table_name;
Table does not exist	Wrong table name or database not selected	Ran USE classicmodels; and SHOW TABLES;
Safe Update Mode Error	MySQL Workbench safe update mode prevents deleting or updating rows unless the WHERE condition uses a key column. Since amount is not a key column, MySQL blocked the delete operation.	Temporarily Disable Safe Update Mode

Overall Result

The SQL assignment was mostly completed successfully. Out of the executed queries, almost all returned the expected results. The only major issue was the delete operation, which failed because MySQL Workbench safe update mode was enabled.

The assignment demonstrates:

- 1. Database selection using USE
- 2. Filtering using WHERE
- 3. Joining related tables
- 4. Aggregation using COUNT, AVG, and SUM
- 5. Grouping using GROUP BY
- 6. Filtering grouped data using HAVING
- 7. Subqueries
- 8. CTEs using WITH
- 9. Ranking using window functions
- 10. View creation
- 11. Update operation
- 12. Delete operation debugging

Possible Improvements

Improvement	Why It Helps
More Detailed Schema Exploration	This would make the report easier to understand from a database design perspective.
More CASE-Based Data Transformation	This would show stronger use of SQL for cleaning and transforming raw business data.
Better Handling of NULL Values	This would improve the data-cleaning part of the assignment.
More Business Insights	This would make the assignment feel more like a real business analytics report.
Use of LEFT JOIN for More Complete Results	This would help identify inactive customers.
Safer Update and Delete Practices	This would make the report safer and more professional.

Conclusion

The Classic Models SQL assignment was completed using MySQL Workbench. The queries helped analyze customer, employee, order, payment, and office-related data. Most tasks were executed successfully, and the encountered errors were identified and solved. The final issue related to safe update mode can be fixed by disabling safe updates temporarily or by using a key column in the WHERE clause.